智能通信的人工智能基础 1：神经网络训练与 CNN

廖振宇

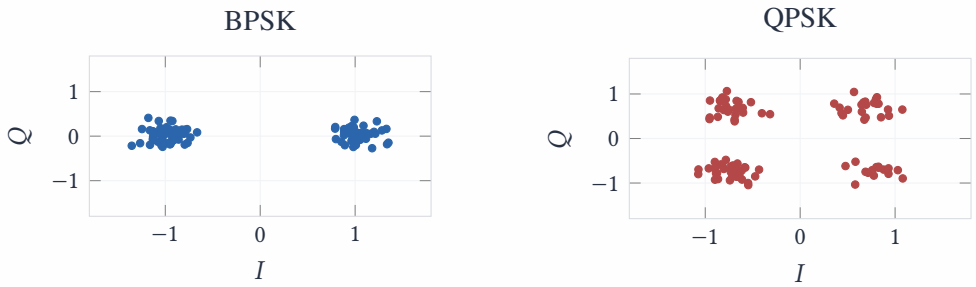
电子信息与通信学院
华中科技大学
2026 年秋季学期

1
神经网络训练

1 课程安排

讲次	上课时间	核心问题
第一讲	周二	神经网络怎样从带标签的数据中学习?
	周五	CNN 如何利用数据中的局部结构?
第二讲	周二	怎样生成样本，怎样让多台设备共同训练?
	周五	数据不能集中时如何训练，如何用 AI 辅助决策?

1 智能通信例子：从接收信号识别调制方式

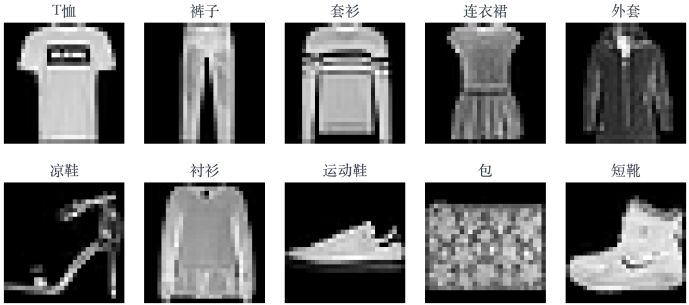


结论

噪声、相位偏移与信道失真会改变观测，AI 模型需从观测/数据中提取有助分类的特征

参考：O'Shea 等，Convolutional Radio Modulation Recognition Networks，2016

1 计算机视觉例子：自动化服饰识别



Fashion-MNIST 图像；每张为 28×28 灰度图

- 鞋底与轮廓可以提供分类信息
- 网络需要自动从样本中学习特征，以区分类别

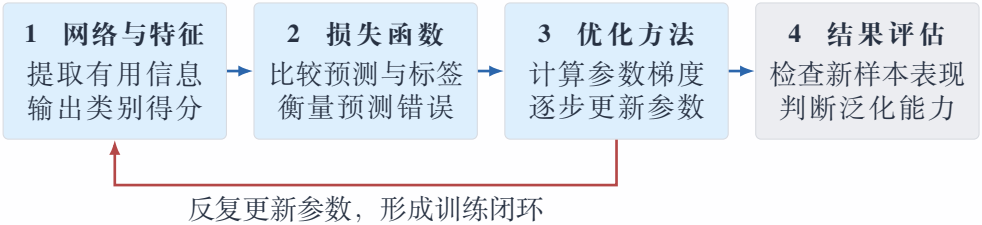
1 视觉分类和无线信号识别的统一视角

要素	服饰分类	调制识别
观测	像素组成的图像	有时间顺序的 I/Q 采样
标签	鞋、包、上衣等类别	BPSK、QPSK 等调制
干扰变化	位置、形状和背景	噪声、相位、频偏和信道
泛化	新图像仍可识别	新采集记录仍可识别

结论

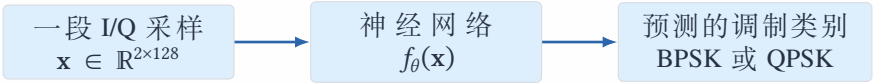
共同点：从（高维）观测预测类别，只是物理含义不同

1 AI 涉及哪些环节？



环节	本讲例子
网络与特征	多层感知机 MLP 与 CNN，从图像像素或 I/Q 采样中学习表示
损失与优化	交叉熵衡量分类错误，反向传播计算梯度，优化器更新参数
结果评估	准确率、类别混淆与泛化，判断模型是否学会规律

1 监督学习：输入、标签与输出



- 先考虑二分类：标签 $y = 0$ 对应 BPSK，$y = 1$ 对应 QPSK
- 训练时使用包含真实标签的成对样本 $(\mathbf{x}, y)$，预测时只有新数据 $\mathbf{x}'$
- 参数 θ 是需要学习的权重与偏置

思考

如果任务改为信道估计，输出应该是（离散）类别，还是连续的信道系数？

1 准备工作：数据划分

数据集	用途	通信场景中的检查
训练集	更新模型参数	计划用于训练的调制、信噪比与信道条件
验证集	选择超参数（hyperparameter，如学习率）	与训练集独立
测试集	最后报告泛化表现	另设未见设备或未见信道的测试

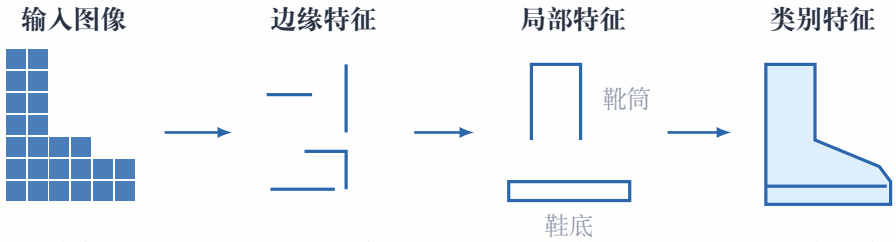
1 课堂判断：随机切窗为何可能泄漏？

- **输入：**无线接收机的一次连续 I/Q 采集记录 $r[n]$
- **任务：**对截取的 128 点窗口 $\mathbf{x} \in \mathbb{R}^{2\times 128}$，识别调制方式为 BPSK 还是 QPSK

$$\underbrace{\mathbf{x}_{\text{train}} = r[0 : 128]}_{\text{训练窗口}} \qquad \underbrace{\mathbf{x}_{\text{test}} = r[64 : 192]}_{\text{测试窗口}}$$

- 两个窗口共享 64 个采样点，且属于同一次采集
- 测试时在“考原题”

1 为什么需要深度？从边缘、局部到识别靴子



- 靴子与运动鞋都可能有鞋底，高靴筒与鞋底的组合提供进一步的分类
- 后一层复用前一层得到的表示，逐步组合出更复杂的特征
- 如果每层都只做线性变换，能得到这样的表达能力吗?

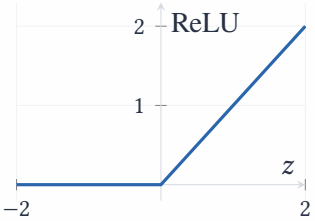
1 非线性激活：让多层深度结构有意义

- 只叠加线性层**不增加表达能力**：两个线性层可以合并

$$W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$$

- 加入非线性

$$\mathrm{ReLU}(z) = \max(0, z)$$

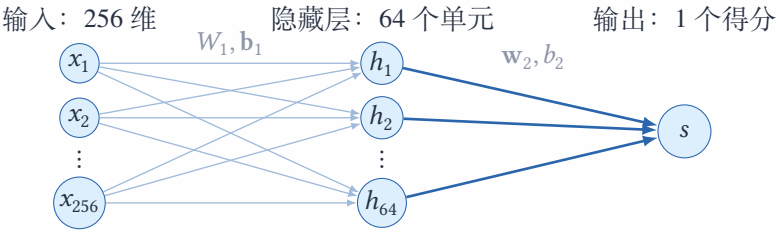


结论

非线性使模型可以表示更复杂的输入—输出关系，从而解决更复杂的问题

- 把线性变换与 ReLU 组合起来，就得到下面的多层感知机

1 多层感知机：从输入数据得到得分



$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x}_{\mathrm{flat}} + \mathbf{b}_1) \in \mathbb{R}^{64}, \qquad s = \mathbf{w}_2^{\mathsf{T}}\mathbf{h} + b_2 \in \mathbb{R}$$

- s 越大越倾向于判断为 QPSK；下一步：将得分 s 转换为概率估计 p

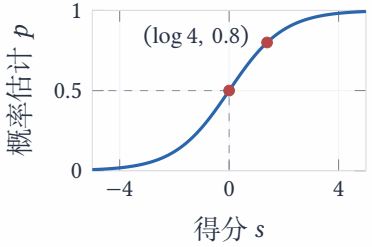
二分类：$y = 0$ 为 BPSK，$y = 1$ 为 QPSK

1 Sigmoid 非线性：把二分类得分转为概率估计

- 输入 x 经过线性 + 非线性变换，形成分类得分 s：
  $\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x}_{\text{flat}} + \mathbf{b}_1)$　　$s = \mathbf{w}_2^{\top}\mathbf{h} + b_2$
- 得分 s 经 Sigmoid 函数形成概率估计

$$p = \mathrm{Sigmoid}(s) = \frac{1}{1 + \exp(-s)}$$

- $\widehat{P}(y = 1 \mid \mathbf{x}) = p$，$\widehat{P}(y = 0 \mid \mathbf{x}) = 1 - p$
  - $s = 0$ 时，两类概率估计均为 0.5
  - $s = \log 4$ 时，QPSK 的概率估计为 0.8
  - 取阈值 0.5：$p \geq 0.5$ 时预测 QPSK



二分类交叉熵损失函数

$L = -y \log p - (1 - y)\log(1 - p)$，$y \in \{0, 1\}$　　真实类别概率越大，损失越小

1 从二分类到多分类：one-hot、Softmax 与交叉熵

1. **one-hot 标签（独热编码）**
   K 个互斥类别，真实类别为 $y \in \{1, \dots, K\}$

$$t_k = \begin{cases} 1, & k = y \\ 0, & k \neq y \end{cases}$$

2. **Softmax：将 K 个得分转为概率**

$$p_k = \frac{\exp(s_k)}{\sum_{j=1}^{K} \exp(s_j)} \qquad p_k > 0, \quad \sum_{k=1}^{K} p_k = 1$$

3. **交叉熵损失**
   $L(\mathbf{t}, \mathbf{p}) = -\sum_{k=1}^{K} t_k \log p_k = -\log p_y$
   one-hot 标签使求和只留下真实类别的一项

例：真实类别为 QPSK

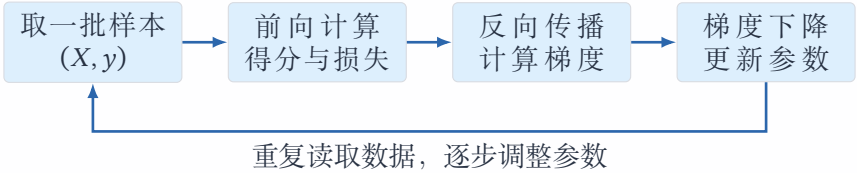
类别顺序为 BPSK、QPSK、16-QAM

1. 标签 $y = 2$
   对应 one-hot：$\mathbf{t} = (0, 1, 0)$
2. 得分 $\mathbf{s} = (0, \log 8, 0)$
   归一化得到 $\mathbf{p} = (0.1, 0.8, 0.1)$
3. 交叉熵只取 QPSK 对应的概率
   $L = -\log 0.8 \approx 0.223$

1 训练闭环：反向传播与梯度下降

- **训练目标：**调整参数 θ，最小化 N 个训练样本上的平均损失

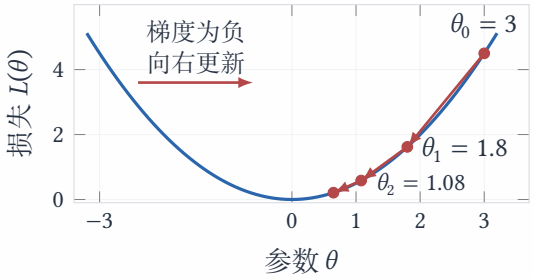
$$\min_{\theta} \mathscr{L}(\theta) \qquad \mathscr{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(\mathbf{x}_i), y_i)$$



$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L_{\text{batch}}$$

- **梯度下降（gradient descent）**：沿负梯度方向更新参数
- 反向传播计算梯度，学习率 η 控制每次更新的幅度

1 梯度下降：沿负梯度方向走向较低的损失



$$L(\theta) = \frac{1}{2}\theta^2 \qquad \frac{\mathrm{d}L}{\mathrm{d}\theta} = \theta$$

$$\theta_{t+1} = \theta_t - \eta\theta_t$$

- 梯度为正，向左更新
- 取 $\eta = 0.4$，从 $\theta_0 = 3$ 出发

$$3 \longrightarrow 1.8 \longrightarrow 1.08$$

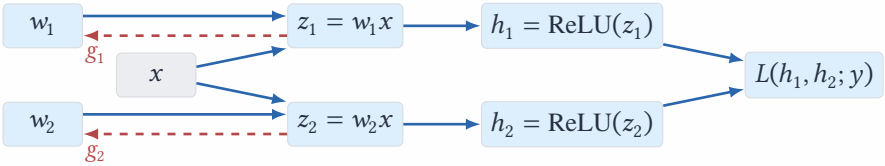
学习率控制步幅

沿负梯度方向更新，步幅过大仍可能跨过最低点，甚至发散

该例中 $0 < \eta < 2$ 时收敛

1 反向传播：沿计算图使用链式法则

- MLP：同一输入 x，两个神经元分别使用独立权重 w_1, w_2



$$\frac{\partial L}{\partial w_i} = g_i = \frac{\partial L}{\partial h_i} \mathbf{1}_{z_i > 0} \, x \qquad (i = 1, 2)$$

- 独立权重分别更新；多条路径汇入同一参数时，贡献相加
- 单神经元平方损失 $L = \frac{1}{2}(h - y)^2$ 给出 $\frac{\partial L}{\partial w} = (h - y)\mathbf{1}_{z>0} x$

蓝色为前向依赖，红色为反向传播的梯度贡献

参考：Dive into Deep Learning，Forward Propagation, Backward Propagation, and Computational Graphs

1 例子：一个回归神经元的参数更新

$$x = 2 \qquad y = 3 \qquad w = 1 \qquad \eta = 0.1$$

1. **前向计算：** $h = \mathrm{ReLU}(wx) = \mathrm{ReLU}(1 \times 2) = 2$，$L = \frac{1}{2}(h - y)^2 = \frac{1}{2}(2 - 3)^2 = 0.5$
2. **反向传播：** $\partial L / \partial w = (2 - 3) \times 1 \times 2 = -2$
3. **更新参数：** $w' = 1 - 0.1 \times (-2) = 1.2$
4. **再看损失：** $h' = 2.4$，$L' = \frac{1}{2}(2.4 - 3)^2 = 0.18$

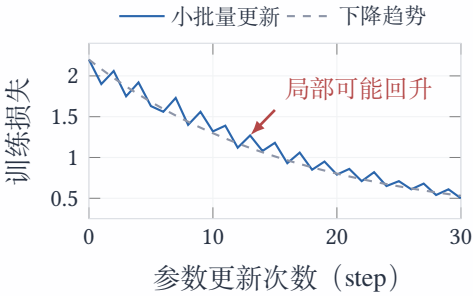
思考

若将学习率改为 $\eta = 1$，这一步更新后损失还会下降吗？

1 小批量梯度与训练轮次

$$L_B(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} L(f_\theta(\mathbf{x}_i), y_i) \qquad \theta \leftarrow \theta - \eta \nabla_\theta L_B$$

- 一个 step：使用一个批次（mini-batch）更新一次参数
- 一个 epoch：遍历一次训练集
- 320 个样本、批量（batch size）32，每个 epoch 有 10 次更新
- 小批量节省单次计算，梯度带有采样波动；但总体下降



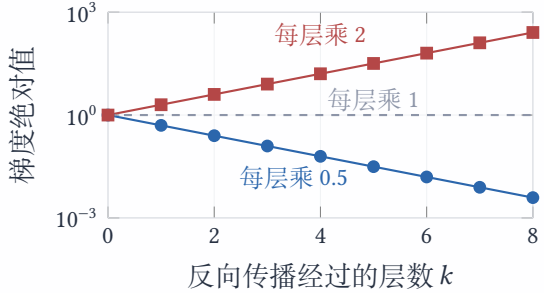
1 训练代码与四个环节对应

```
model.train()
for x, y in loader:                  # 1. batch
    optimizer.zero_grad()
    logits = model(x)                # 2. forward
    loss = criterion(logits, y)      # 2. loss
    loss.backward()                  # 3. backward
    optimizer.step()                 # 4. update
```

1. **取一批样本：**读取输入 `x` 与标签 `y`
2. **前向计算：**得到得分 `logits`，再计算损失 `loss`
3. **反向传播：**`backward()` 计算梯度，每批开始前清除旧梯度
4. **参数更新：**`step()` 按优化器规则调整参数

参考：PyTorch 官方教程，Optimizing Model Parameters

1 深层网络训练中的挑战

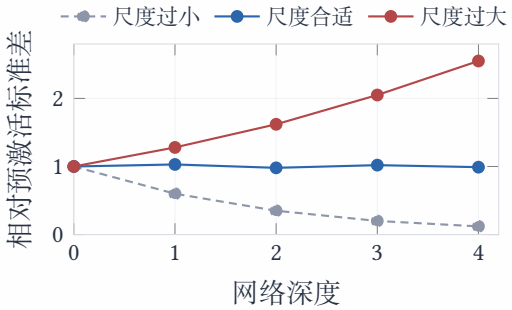


- **初始化**使早期激活与梯度处于合理尺度
- **归一化**调整中间表示的尺度，注意训练与推理模式
- **残差连接**提供较直接的信息与梯度路径

链式法则中的多层相乘

从梯度 1 出发，连乘 8 次后　$0.5^8 \approx 0.0039$，而 $2^8 = 256$

1 初始化：让各层从合适的尺度开始



- **随机设置权重**，让不同隐藏单元从不同起点学习
- 尺度过小或过大，可能使信号逐层衰减或放大
- 依据输入宽度和激活函数选择权重尺度

ReLU 层常用 He 初始化

$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$　　n_{in} 为每个神经元的输入数，偏置可置零

预激活指非线性变换之前的值　　参考：He 等，2015

1 归一化：以 Batch Normalization 为例

- 对同一隐藏特征在一批样本中的取值 $z_1, \dots, z_m$，调整均值与尺度

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z_i \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu_B)^2$$

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \qquad h_i = \gamma \hat{z}_i + \beta$$

- γ, β 为可学习的尺度与偏移，$\varepsilon > 0$ 避免除零

训练时

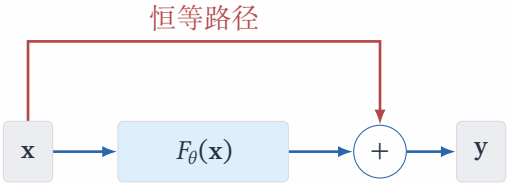
- 使用当前批次的均值与方差
- 更新运行均值与运行方差

验证与推理时

- 使用训练期间保存的统计量
- 保留学到的 γ, β，切换到 `eval` 模式

参考：Ioffe 与 Szegedy，2015

1 残差连接：保留输入，并学习修正



$$\mathbf{y} = \mathbf{x} + F_\theta(\mathbf{x})$$

- 学习相对输入的改变量
- $F_\theta(\mathbf{x}) = \mathbf{0}$ 时保留输入
- 两条分支的对应元素相加，尺寸须相同

恒等路径让梯度直接传回，有助于训练深层网络

$$\nabla_{\mathbf{x}} L = \underbrace{\nabla_{\mathbf{y}} L}_{\text{沿恒等路径直接传回}} + \underbrace{J_F^{\mathsf{T}} \nabla_{\mathbf{y}} L}_{\text{经过残差分支传回}} \qquad J_F = \frac{\partial F_\theta}{\partial \mathbf{x}}$$

参考：He 等，2016

1 评估分类结果：准确率与混淆矩阵

$$\text{准确率（Accuracy）} = \frac{\text{预测正确的样本数}}{\text{评估样本总数}}$$

	预测 BPSK	预测 QPSK
真实 BPSK	45	5
真实 QPSK	10	40

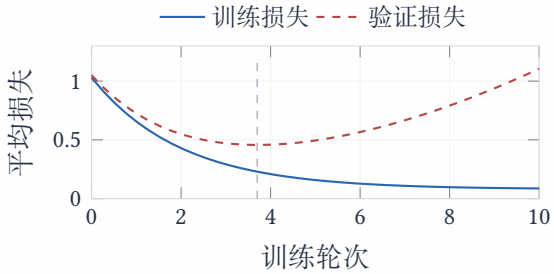
$$\text{准确率} = \frac{45 + 40}{45 + 5 + 10 + 40} = \frac{85}{100} = 85\%$$

- 对角线计数对应正确分类
- 非对角线说明混淆发生在哪些类别之间
- QPSK 的识别正确比例为 $40/50 = 80\%$

结论

总体准确率概括表现，混淆矩阵说明错误的组成

1 从训练损失到验证损失



- **训练损失**：训练集上的平均损失，参与参数优化
- **验证损失**：独立验证集上的平均损失，不参与梯度更新
- 在验证损失较低处选择模型（早停，early stopping）

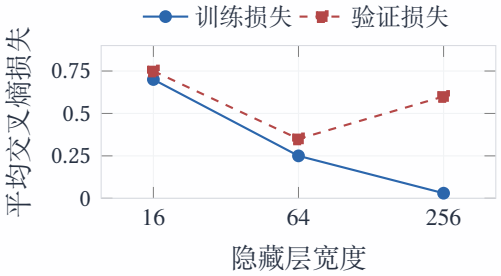
结论

训练损失持续下降 ≠ 验证损失继续下降

验证集用于选择停止轮次，测试集留到最后评估

1 模型容量与泛化：训练损失低还不够

- 调制识别例子：分别训练不同隐藏层宽度的 MLP



宽度	训练损失	验证损失
16	0.70	0.75
64	0.25	**0.35**
256	**0.03**	0.60

- **欠拟合：**未充分学到数据规律，训练与验证损失都较高
- **过拟合：**过度适应训练样本，训练损失低而验证损失较高

泛化：对未参与训练的新样本仍能有效预测

训练损失更低不保证泛化更好，关注独立样本上的表现

1 可能的问题

观察	优先检查
很小的训练子集也拟合不了	标签与输入对应、损失实现、梯度、学习率
训练下降，验证明显变差	数据量与分布、模型容量、正则化、早停
随机划分很好，新场景很差	采集记录泄漏、设备偏差、训练场景覆盖
总体准确率高，低信噪比很差	按信噪比和类别分别报告表现

结论

先区分优化是否有成效，再判断模型能否泛化

1 课堂练习

1. 对 $h = \mathrm{ReLU}(wx)$，取 $x = 2, y = 3, w = 1, \eta = 1$，损失为 $L = \frac{1}{2}(h - y)^2$，求更新后的 w'、输出和损失
2. 200 个训练样本、批量 20，训练 5 个 epoch，总共有多少次参数更新？
3. 将同一次采集的 I/Q 序列切成重叠窗口后随机划分，可能产生什么问题，怎样修改？

1 课堂练习：答案与解释

1. 对 $h = \mathrm{ReLU}(wx)$，取 $x = 2, y = 3, w = 1, \eta = 1$，损失为 $L = \frac{1}{2}(h - y)^2$，求更新后的 w'、输出和损失
   答案： $w' = 1 - 1 \times (-2) = 3$，$h' = 6$，$L' = \frac{1}{2}(6 - 3)^2 = 4.5$
   解释： 梯度为 $(2 - 3) \times 1 \times 2 = -2$，步幅过大使损失从 0.5 增至 4.5

2. 200 个训练样本、批量 20，训练 5 个 epoch，总共有多少次参数更新？
   答案： $(200/20) \times 5 = 50$ 次更新
   解释： 每个批次更新一次，每轮 10 次，共 5 轮

3. 将同一次采集的 I/Q 序列切成重叠窗口后随机划分，可能产生什么问题，怎样修改？
   答案： 可能发生数据泄漏，应先按采集记录分组划分，再在各组内切窗
   解释： 重叠窗口共享采样与环境信息，会使测试结果过于乐观

1 神经网络训练：总结

学习目标

- 用自己的话区分前向传播、反向传播与参数更新
- 解释训练损失下降和泛化的关系
- 指出调制识别任务中的输入和标签

思考

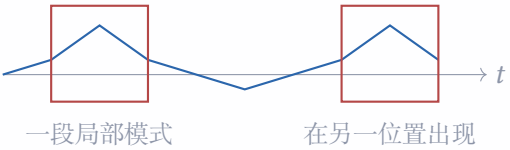
一段 I/Q 信号包含相邻采样之间的局部结构，展平后的 MLP 怎样利用它？

卷积神经网络 CNN

用**局部连接**与**权重共享**，把（时间）邻接关系写入网络结构

2
卷积神经网络

2 局部结构与权重共享



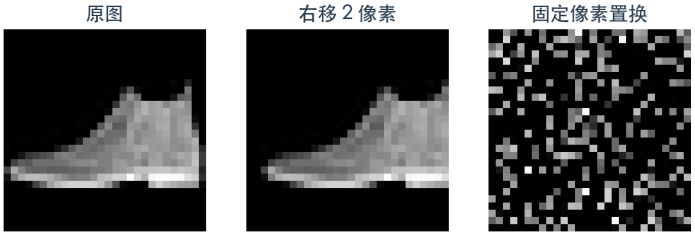
- **局部连接**：仅读取短时间窗口
- **权重共享**：相同规则，扫描序列不同位置
- **多个卷积核**：学习不同局部模式

结论

CNN 考虑“相邻采样有关联、局部规律可复用”作为结构假设

同样的思想适用于图像上的二维邻域

2 图像中的局部结构



对同一图像作平移与固定像素置换 P，说明**局部邻接关系**

- **平移**保留大部分局部邻接关系
- **置换**保留像素值但破坏邻接关系
- 卷积依赖输入排列所表达的结构

2 一维卷积的例子

$$\mathbf{x} = (1,\ 2,\ 3,\ 4)^{\mathsf{T}} \qquad \mathbf{w} = (1,\ 2)$$

窗口	同一组权重的加权和	输出
$[1,2]$	$1\times 1 + 2\times 2$	5
$[2,3]$	$1\times 2 + 2\times 3$	8
$[3,4]$	$1\times 3 + 2\times 4$	11

结论

滑动窗口只改变读取的位置，核的权重保持**相同**

核宽 2、步幅 1、不补零、无偏置；采用不翻转卷积核计算规则

2 一维卷积的矩阵表示：局部连接与权重共享

- 单通道输入 $\mathbf{x} = (x_1, x_2, x_3, x_4)^\mathsf{T}$，卷积核 $\mathbf{w} = (a, b)$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \underbrace{\begin{bmatrix} a & b & 0 & 0 \\ 0 & a & b & 0 \\ 0 & 0 & a & b \end{bmatrix}}_{C(\mathbf{w})} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} ax_1 + bx_2 \\ ax_2 + bx_3 \\ ax_3 + bx_4 \end{bmatrix}$$

- **局部连接：**每行只使用相邻两个输入，框内对应第一行的两个连接
- **权重共享：**a、b 在各行**重复**，只有 2 个可学习参数

上一页的例子

$\mathbf{x} = (1, 2, 3, 4)^\mathsf{T}$，$\mathbf{w} = (1, 2) \implies \mathbf{y} = (5, 8, 11)^\mathsf{T}$

核宽 2、步幅 1、不补零、无偏置

2 二维卷积：局部窗口与输出一一对应

输入 X 卷积核 K 输出 Y

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 1 \end{bmatrix}$$

$$Y_{11} = 1\times 1 + 2\times 0 + 0\times 0 + 1\times(-1) = 0$$

$$Y_{12} = 2 - 3 = -1 \qquad Y_{21} = 0 - 1 = -1 \qquad Y_{22} = 1 - 0 = 1$$

- 左上角 2×2 窗口与核对应相乘并求和，得到输出 Y_{11}
- 同一个卷积核继续滑动，得到其余三个输出

输入 3×3，核 2×2，步幅为 1、不补零、无偏置

2 卷积得到特征图，还要再做非线性变换

$$\text{MLP:}\quad \mathbf{h} = \mathrm{ReLU}(W\mathbf{x})$$

$$\text{CNN:}\quad \mathbf{h} = \mathrm{ReLU}(W_{\mathrm{conv}}\mathbf{x}),\qquad W_{\mathrm{conv}} = C(\mathbf{w})$$

- W 为全连接层的权重矩阵，各连接权重可独立学习
- W_{conv} 受约束：**局部连接**产生零元素，**权重共享**使相同系数重复出现

$$Z = \begin{bmatrix} 0 & -1 \\ -1 & 1 \end{bmatrix} \xrightarrow{\mathrm{ReLU}} H = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

结论

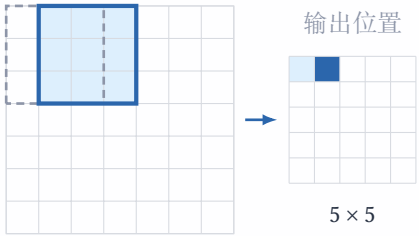
两者都是线性变换后接逐元素 ReLU；CNN 只是有约束的线性变换

2 步幅 stride：卷积窗口每次移动多远？

- 7×7 输入，3×3 卷积核，不补零

步幅 $s = 1$

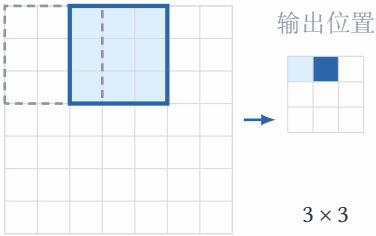
右移 1 格



完整扫描：$5 \times 5 = 25$ 个输出

步幅 $s = 2$

右移 2 格

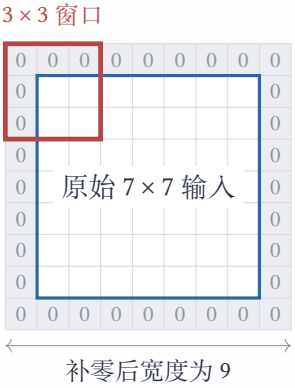


完整扫描：$3 \times 3 = 9$ 个输出

- 步幅 s 控制移动**间隔**；每个合法窗口产生一个输出位置

参考：CS231n：Convolutional Neural Networks，Spatial Arrangement

2 补零 zero-padding：在输入四周扩展边界



- $p = 1$ 表示上、下、左、右每侧各补 1 层零
- 补零后尺寸为 9×9，边界窗口会读到补入的零
- 固定 3×3 卷积核、步幅 $s = 1$

$$p = 0: \quad 7 \times 7 \longrightarrow 5 \times 5$$

$$p = 1: \quad 9 \times 9 \longrightarrow 7 \times 7$$

- 一维信号左右各补 1 个零：$[0, x_1, \ldots, x_L, 0]$，长度变为 $L + 2$

参考：CS231n：Convolutional Neural Networks，Zero-padding

2 步幅与补零控制输出长度

$$L_{\text{out}} = \left\lfloor \frac{L + 2p - k}{s} \right\rfloor + 1$$

输入长度 L	核宽 k	补零 p、步幅 s	输出长度
128	5	$p = 0,\ s = 1$	124
128	5	$p = 2,\ s = 1$	128
128	5	$p = 2,\ s = 2$	64

结论

步幅减少输出位置，补零保留边界附近的计算位置

注：左右对称补零

2 权重共享减少需要学习的参数

- C_{in} 是输入通道数，C_{out} 是输出特征通道数
- 本例 I/Q 共 2 个输入通道，16 个卷积核各读取两个通道并生成一个输出通道

输入 2×128，输出 16×128；卷积取 $k = 5, p = 2, s = 1$

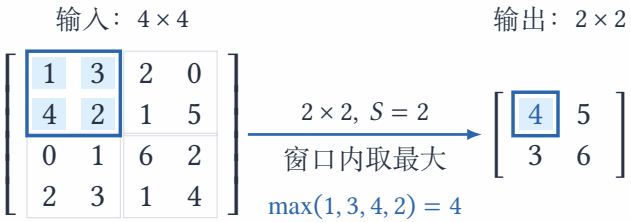
结构	参数计算（含偏置）	参数量
全连接层	$(2 \times 128) \times (16 \times 128) + 16 \times 128$	526 336
卷积层，$k = 5$	$16 \times (2 \times 5 + 1)$	176

$$\text{卷积参数量} = C_{\text{out}}(C_{\text{in}}k + 1)$$

结论

共享参数不随输入长度增长，但计算量仍随输出位置数增长

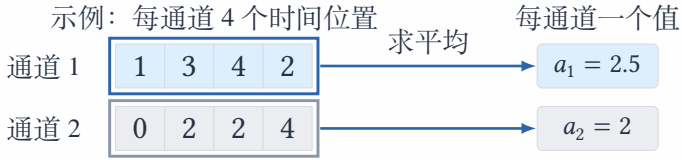
2 最大池化 Max Pooling：保留局部最强响应



- 池化通常逐通道进行，不混合不同通道
- 无可学习参数
- 压缩空间大小，同时丢失窗口内的精确位置信息

2 平均池化：从局部平均到全局平均

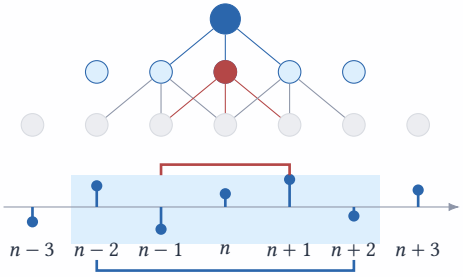
- 平均池化在窗口内取算术平均，如上一页：$(1+3+4+2)/4=2.5$
- **全局平均池化** (GAP, Global Average Pooling)：窗口覆盖该通道的全部位置，每个通道保留一个平均值



结论

不混合通道、无可学习参数

2 深层特征可以覆盖更长的时间范围



结论

小卷积核逐层**组合局部信息**，深层单元的**感受野**随之扩大

两层核宽 3、步幅 1 的卷积，忽略边界　　感受野表示可能影响该单元的输入范围

2 感受野：逐层追溯一个输出依赖哪些输入

$$r_l = r_{l-1} + (k_l - 1) j_{l-1}, \qquad j_l = s_l j_{l-1}$$

层	核宽与步幅	感受野 r	输入间隔 j
输入	—	1	1
卷积 1	$k = 3, s = 2$	3	2
卷积 2	$k = 3, s = 1$	7	2

结论

第二层相邻位置跨过两个原始采样点，因此感受野增加 4

从 $r_0 = j_0 = 1$ 开始，忽略边界截断

2 调制识别 CNN 结构

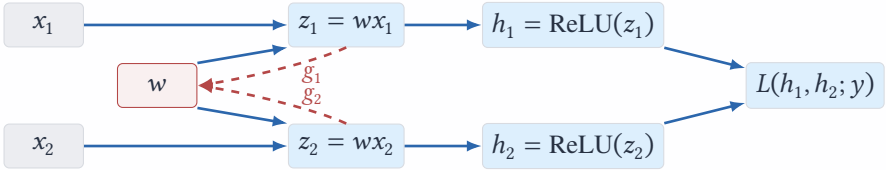


层	设置	单样本尺寸	参数量
输入	I/Q 采样	2×128	0
Conv1 + ReLU	$2 \to 16,\ k=5, p=2, s=1$	16×128	176
Conv2 + ReLU	$16 \to 32,\ k=5, p=2, s=2$	32×64	2 592
全局时间平均池化	对每个通道沿时间求平均	32	0
线性输出层	$32 \to 3$	3 个得分	99
合计			2 867

- MLP：$\mathbf{h} = \mathrm{ReLU}(W\mathbf{x})$；CNN：$\mathbf{h} = \mathrm{ReLU}(W_{\mathrm{conv}}\mathbf{x})$
- $W_{\mathrm{conv}} = C(\mathbf{w})$ 具有局部连接与权重共享的约束；省略偏置

2 CNN 的训练闭环与 MLP 相同

- CNN 的最小示意：两个时间位置 x_1, x_2 复用同一个卷积权重 w

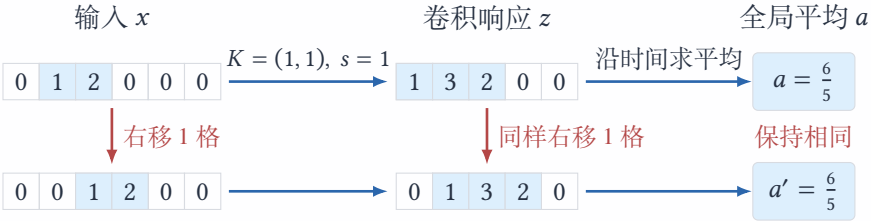


$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial h_1}\mathbf{1}_{z_1>0}x_1}_{g_1} + \underbrace{\frac{\partial L}{\partial h_2}\mathbf{1}_{z_2>0}x_2}_{g_2}$$

- **区别：** MLP 的独立权重分别更新；CNN 的共享权重先汇总各位置的贡献
- **相同：** 前向计算得分与损失，反向传播求梯度，优化器更新参数并检验泛化

取核宽 1、单输入和输出通道、无偏置；更大的卷积核中，每个系数同样跨位置共享

2 平移等变与分类不变



- **等变：** 输入移动，特征响应跟着移动；步幅为 1、忽略边界时成立
- **不变：** 最终类别预测不变；本例中只读全局平均 a 的分类头会得到相同预测
- 边界、下采样与截窗仍会影响结果，池化不保证一般情况下的分类不变

思考

信号移动一个采样后，CNN 的调制预测一定不变吗?

2 评价 CNN：预测质量与计算代价

- **预测质量**看准确率与混淆矩阵，**参数量**对应权重和偏置的存储
- **运算量**统计乘加等操作的次数，**推理延迟**还需在目标设备上测量

同样输入 2×128、输出 3 个类别得分；内部结构不同，MLP 先将输入展平

网络	参数量（含偏置）	每个样本的乘加次数
MLP：$256 \to 64 \to 3$	$64(256+1)+3(64+1)$ $= \mathbf{16\,643}$	$256 \times 64 + 64 \times 3$ $= \mathbf{16\,576}$
前面的 CNN	$176 + 2\,592 + 99$ $= \mathbf{2\,867}$	$20\,480 + 163\,840 + 96$ $= \mathbf{184\,416}$

卷积乘加次数：输出长度 × 输出通道数 ×（输入通道数 × 核宽）；表中仅计卷积与全连接

CNN 权重共享节省参数，不（一定）减少计算

本例 CNN 的参数更少，乘加次数却约为 MLP 的 11.1 倍

2 课堂练习

1. 输入为 2×128，输出通道为 8，$k = 3, p = 1, s = 2$，输出尺寸和含偏置的参数量各是多少?
2. 两层核宽 3、步幅 1 的普通卷积，忽略边界，第二层的感受野是多少?
3. 将第一题输入长度改为 256，其余设置不变，输出尺寸和参数量分别怎样变化?

2 答案

1. 输入为 2×128，输出通道为 8，$k = 3, p = 1, s = 2$，输出尺寸和含偏置的参数量各是多少？

   答案： $L_{\text{out}} = \lfloor (128 + 2 - 3)/2 \rfloor + 1 = 64$，输出为 8×64

   参数量为 $8(2 \times 3 + 1) = 56$

   解释： 每个输出通道含 2×3 个权重与 1 个偏置，权重在各位置共享

2. 两层核宽 3、步幅 1 的普通卷积，忽略边界，第二层的感受野是多少？

   答案： $3 + (3 - 1) \times 1 = 5$ 个输入位置

   解释： 第二层组合第一层相邻的 3 个输出，使覆盖范围在 3 的基础上增加 2

3. 将第一题输入长度改为 256，其余设置不变，输出尺寸和参数量分别怎样变化？

   答案： $L_{\text{out}} = \lfloor (256 + 2 - 3)/2 \rfloor + 1 = 128$，输出为 8×128，参数量仍为 56

   解释： 输入变长增加卷积核滑动的位置数，但不增加需要学习的权重和偏置

2 第一讲总结

1. CNN 如何利用信号的结构?
2. 交叉熵解决什么问题?
3. 反向传播和优化器分别做什么?
4. 训练损失很低是否足够?

思考

如果缺少某些场景的信号数据，或者数据分散在多座基站，训练怎样继续?

2 进一步学习：网络、优化与卷积

- **计算图与反向传播**
  Dive into Deep Learning：前向传播、反向传播与计算图
  阅读目标：指出前向依赖关系，解释链式法则如何传递梯度
- **训练更新的四个环节**
  PyTorch 官方教程：Optimizing Model Parameters
  阅读目标：区分计算输出、计算损失、计算梯度与更新参数
- **卷积网络的结构**
  Stanford CS231n：Convolutional Neural Networks
  阅读目标：说明局部连接、权重共享、池化和感受野的作用